

```
=====
                        BINARY SEARCH PROGRAM
                        in C Language
=====
```

```
PROGRAM DESCRIPTION:
```

This program implements the Binary Search algorithm to find a specific element in a sorted array. Binary search is an efficient searching technique with a time complexity of $O(\log n)$.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[100], n, key;
    int low, high, mid;
    int found = 0;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements in sorted order:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    low = 0;
    high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == key) {
            printf("Element found at position %d\n", mid + 1);
            found = 1;
            break;
        }
        else if (arr[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    if (!found) {
        printf("Element not found\n");
    }

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. VARIABLE DECLARATIONS:

- arr[100]: Array to store elements (max 100 elements)

- n: Number of elements
- key: Element to search for
- low: Starting index
- high: Ending index
- mid: Middle index
- found: Flag to track if element is found

2. INPUT PHASE:

- Takes number of elements from user
- Accepts sorted array elements
- Accepts the element to search

3. BINARY SEARCH ALGORITHM:

- Initializes low = 0, high = n-1
- Repeatedly divides search space in half:
 - * Calculates mid = (low + high) / 2
 - * Compares arr[mid] with key
 - * If equal: Element found, break
 - * If arr[mid] < key: Search right half (low = mid + 1)
 - * If arr[mid] > key: Search left half (high = mid - 1)
- Continues until element is found or search space is exhausted

4. OUTPUT:

- Displays position if element is found
- Displays message if element is not found

HOW TO COMPILE & RUN

Compilation:

```
gcc -o binary_search binary_search.c
```

Execution:

```
./binary_search
```

Example Input:

```
Enter the number of elements: 5
Enter 5 elements in sorted order:
10 20 30 40 50
Enter the element to search: 30
```

Example Output:

```
Element found at position 3
```

KEY FEATURES

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Precondition: Array must be sorted in ascending order

Advantages:

- Much faster than linear search for large datasets
- Efficient $O(\log n)$ time complexity
- Suitable for sorted data

Limitations:

- Array must be pre-sorted
- Not suitable for unsorted data
- Takes more time for very small arrays (overhead)